OS – Unit 4
Orange Problem

Name: Pranav Chandrasekar

SRN: PES1UG24CS332

Screenshot 1A:

```
pc@pc-virtual-machine:~/PES1UG24CS332-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

Screenshot 1B:

```
pc@pc-virtual-machine:~/PES1UG24CS332-pes-vcs$ find .pes/objects -type f
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
```

Screenshot 2A:

```
pc@pc-virtual-machine:~/PES1UG24CS332-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

Screenshot 2B:

```
pc@pc-virtual-machine:~/PES1UG24CS332-pes-vcs$ xxd .pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf | head -20
00000000: 626c 6f62 2031 3500 5465 7374 2069 6e74  blob 15.Test int
00000010: 6567 7269 5879 0a                        egriXy.
```

Screenshot 3A:

```
pc@pc-virtual-machine:~/PES1UG24CS332-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
pc@pc-virtual-machine:~/PES1UG24CS332-pes-vcs$ nano file1.txt
pc@pc-virtual-machine:~/PES1UG24CS332-pes-vcs$ ./pes add file1.txt
pc@pc-virtual-machine:~/PES1UG24CS332-pes-vcs$ nano file2.txt
pc@pc-virtual-machine:~/PES1UG24CS332-pes-vcs$ ./pes add file2.txt
pc@pc-virtual-machine:~/PES1UG24CS332-pes-vcs$ ./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  commit.c
  untracked:  test_objects.c
  untracked:  commit.h
  untracked:  .gitignore
  untracked:  pes.h
  untracked:  Makefile
  untracked:  tree.h
  untracked:  tree.c
  untracked:  test_sequence.sh
  untracked:  index.h
  untracked:  object.c
  untracked:  index.c
  untracked:  test_tree.c
  untracked:  pes.c
  untracked:  README.md
```

Screenshot 3B:

```
pc@pc-virtual-machine:~/PES1UG24CS332-pes-vcs$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776706877 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776702327 6 file2.txt
```

Screenshot 4A:

```
pc@pc-virtual-machine:~/PES1UG24CS332-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
pc@pc-virtual-machine:~/PES1UG24CS332-pes-vcs$ echo "Hello" > hello.txt
pc@pc-virtual-machine:~/PES1UG24CS332-pes-vcs$ ./pes add hello.txt
pc@pc-virtual-machine:~/PES1UG24CS332-pes-vcs$ ./pes commit -m "Initial commit"
Committed: 257d9f8dc44c... Initial commit
pc@pc-virtual-machine:~/PES1UG24CS332-pes-vcs$ echo "World" > world.txt
pc@pc-virtual-machine:~/PES1UG24CS332-pes-vcs$ ./pes add world.txt
pc@pc-virtual-machine:~/PES1UG24CS332-pes-vcs$ ./pes commit -m "Add world"
Committed: fe835b5f4f95... Add world
pc@pc-virtual-machine:~/PES1UG24CS332-pes-vcs$ echo "goodbye" > bye.txt
pc@pc-virtual-machine:~/PES1UG24CS332-pes-vcs$ ./pes add bye.txt
pc@pc-virtual-machine:~/PES1UG24CS332-pes-vcs$ ./pes commit -m "Fare thee well"
Committed: 90404aed181d... Fare thee well
pc@pc-virtual-machine:~/PES1UG24CS332-pes-vcs$ ./pes log
commit 90404aed181d7f1a44e652966beed1592c08b6ecffece33f678de90f36447a9e
Author: PES User <pes@localhost>
Date:   1776707195

    Fare thee well

commit fe835b5f4f957a5fc16e18759c17d5964d890cca507020912cc60423597ad05d
Author: PES User <pes@localhost>
Date:   1776707164

    Add world

commit 257d9f8dc44ce4087c200c0b21179f57676454cc523fb3edaac3c2381e9c5a98
Author: PES User <pes@localhost>
Date:   1776707136

    Initial commit
```

Screenshot 4B:

```
pc@pc-virtual-machine:~/PES1UG24CS332-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/25/7d9f8dc44ce4087c200c0b21179f57676454cc523fb3edaac3c2381e9c5a98
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/7b/897b5e3b012bdab584142673f3e326d0f87a856507d302aff333058bbeff16
.pes/objects/85/e367b76c2417b47a95f46208835ad8700dd58335171c94a497974be6191900
.pes/objects/90/404aed181d7f1a44e652966beed1592c08b6ecffece33f678de90f36447a9e
.pes/objects/a8/fb75a6e6e80c5e3acedc4067370a15a01156d717352eca10ee3f9d2a66fc49
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/b3/d731c8f5ea38a1118f9300e2dc257dee11461d53f4a1340e6afc4878ebec12
.pes/objects/e0/0c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f
.pes/objects/fe/835b5f4f957a5fc16e18759c17d5964d890cca507020912cc60423597ad05d
.pes/refs/heads/main
```

Screenshot 4C:

```
pc@pc-virtual-machine:~/PES1UG24CS332-pes-vcs$ cat .pes/refs/heads/main
90404aed181d7f1a44e652966beed1592c08b6ecffece33f678de90f36447a9e
pc@pc-virtual-machine:~/PES1UG24CS332-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
```

Final:

```
pc@pc-virtual-machine:~/PES1UG24CS332-pes-vcs$ make test-integration
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


--- First Commit ---
Committed: b97ed006351a... Initial commit

Log after first commit:
commit b97ed006351ad2d0281a15177fd5bc069d69a3defb2b2967d4a8fc6d79987517
Author: Pranav Chandrasekar <PES1UG24CS332>
Date:   1776707491

    Initial commit


--- Second Commit ---
Committed: e3bf8a32bebb... Update file.txt

--- Third Commit ---
Committed: 444452674323... Add farewell

--- Full History ---
commit 444452674323f6e4606aa49c7c2cf7fcae5cb7c096005606f1424bfb4410a137
Author: Pranav Chandrasekar <PES1UG24CS332>
Date:   1776707492

    Add farewell
```

```
    Add farewell

commit e3bf8a32bebb05391ac127c5fe1bce817708680fab92fd85a8e3012bc8e21b51
Author: Pranav Chandrasekar <PES1UG24CS332>
Date:   1776707491

    Update file.txt

commit b97ed006351ad2d0281a15177fd5bc069d69a3defb2b2967d4a8fc6d79987517
Author: Pranav Chandrasekar <PES1UG24CS332>
Date:   1776707491

    Initial commit


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
444452674323f6e4606aa49c7c2cf7fcae5cb7c096005606f1424bfb4410a137

--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/44/4452674323f6e4606aa49c7c2cf7fcae5cb7c096005606f1424bfb4410a137
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/b9/7ed006351ad2d0281a15177fd5bc069d69a3defb2b2967d4a8fc6d79987517
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafeaa3829f61f75b8d10d5350f9
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff
.pes/objects/e3/bf8a32bebb05391ac127c5fe1bce817708680fab92fd85a8e3012bc8e21b51

=== All integration tests completed ===
pc@pc-virtual-machine:~/PES1UG24CS332-pes-vcs$
```

Q5.1 — Branching and Checkout

To implement pes checkout <branch>, the system must update .pes/HEAD to point to the new branch (e.g., ref: refs/heads/<branch>), and ensure the corresponding file in .pes/refs/heads/ contains the correct commit hash. Then, the working directory must be updated to match the tree of that commit: files present in the target commit should be created/overwritten, and files not present should be removed. The index must also be updated to reflect this new snapshot. The complexity arises because this operation modifies multiple layers simultaneously (refs, index, working directory), and must ensure consistency, especially when files differ between branches or when uncommitted changes exist.

Q5.2 — Detecting Dirty Working Directory

A dirty working directory can be detected by comparing the current working directory against the index. For each tracked file, we compare metadata such as modification time and size stored in the index with the current

filesystem state. If these differ, the file has been modified. Additionally, by comparing the index against the target commit's tree (via object store), we can detect whether switching branches would overwrite those changes. If a file is modified locally and also differs in the target branch, checkout must abort to prevent data loss.

Q5.3 — Detached HEAD

In a detached HEAD state, HEAD points directly to a commit hash instead of a branch. New commits made in this state are not referenced by any branch, so they can become unreachable once HEAD moves elsewhere. However, these commits still exist in the object store temporarily. A user can recover them by creating a new branch pointing to that commit (e.g., using the commit hash or reflog in real Git). If no reference is created, these commits may eventually be garbage-collected.

Q6.1 — Garbage Collection Algorithm

To find unreachable objects, we perform a graph traversal starting from all references (branch heads). Using a DFS or BFS, we follow commit objects to their parent commits and associated tree objects, and recursively traverse trees to reach all blobs. All visited hashes are marked as "reachable." A hash set (or bitmap) is ideal for tracking visited objects efficiently. After traversal, any object in .pes/objects not in this set is unreachable and can be deleted. For a repository with 100,000 commits and 50 branches, we would traverse roughly all reachable commits (up to ~100,000) plus their associated trees and blobs, which may total several hundred thousand objects depending on repository size.

Q6.2 — GC vs Concurrent Commit Race Condition

Running garbage collection concurrently with a commit is dangerous because of a race condition: a commit operation may write new objects (blobs/trees) but not yet update the branch reference. If GC runs at this moment, it may see those objects as unreachable and delete them before the commit completes, corrupting the repository. Git avoids this using techniques like write ordering (objects written before refs updated), temporary object protection, and lock files. Additionally, Git may delay deletion (e.g., using grace periods) or perform GC only when no concurrent operations are active.